

SMART STUDY ENVIRONMENT MONITORING & ANALYSIS SYSTEM

Complete Project Report, Final Code, Working Explanation and Viva Preparation

Platform: ESP8266 NodeMCU (ESP-12E) | DHT11 | LDR | HC-SR04 | I2C LCD | LED

Purpose: Monitor a student's study environment, detect presence, track study/break time, calculate an environment score, warn about low light, and display useful recommendations.

1. Project Overview

This is an embedded monitoring system built around the ESP8266 NodeMCU. It reads temperature, humidity, light and distance, processes the readings and presents useful information on a 16×2 I2C LCD and Serial Monitor.

The project demonstrates practical Embedded C/C++ concepts including variables, functions, conditions, GPIO, analog input, sensor interfacing, serial communication and non-blocking timing with `millis()`.

2. Why We Did This Project

- A study area should be comfortable and adequately lit.
- Students may lose track of study and break time.
- Automatic presence detection avoids manual timing.
- Sensor readings can be converted into a simple environment score.
- The project combines hardware and software into a real working embedded application.

3. Objectives

- Measure temperature and humidity with DHT11.
- Measure relative light level with an LDR.
- Detect a nearby person using HC-SR04.
- Turn an LED on when light becomes low.
- Track study time and absence/break time.
- Calculate a score out of 100.
- Display readings and status on LCD.
- Print detailed reports to Serial Monitor.
- Give recommendations and an efficiency rating.

4. Components and Connections

Component	Purpose	NodeMCU connection
ESP8266 NodeMCU	Main controller	—
DHT11	Temperature/humidity	DATA → D4
HC-SR04	Distance/presence	TRIG → D5, ECHO → D6
LDR	Light	A0
LED + 1 kΩ resistor	Low-light indicator	D7
16×2 I2C LCD	Display	SDA → D2, SCL → D1

Important: If using a typical 5 V HC-SR04, protect the ESP8266 ECHO input with a suitable voltage divider/level shifter because ESP8266 GPIO uses 3.3 V logic.

5. How the System Works

DHT11 provides temperature and humidity. The LDR produces an analog light reading through A0. HC-SR04 measures distance; a distance between 2 and 100 cm is treated as person present. The ESP8266 calculates temperature, humidity and light scores, adds a presence score, updates timers, controls the LED, and sends the result to the LCD and Serial Monitor.

The LED uses hysteresis: below 150 the LED turns ON, above 250 it turns OFF. Between those values it keeps its previous state, reducing unwanted rapid blinking.

6. Environment Score

- Temperature: 20–28 °C = 25 points; 18–<20 or >28–30 = 20; 15–<18 or >30–33 = 10; otherwise 5.

- Humidity: 40–60% = 25; 30–<40 or >60–70 = 20; 20–<30 or >70–80 = 10; otherwise 5.
- Light: ≥500 = 25; 200–499 = 18; below 200 = 8.
- Person present = 25 points.
- Maximum = 100 points.

7. Study, Break, Recommendation and Efficiency

When a person is present, studyTime increases every second. When absent, breakTime and absentTime increase. At 300 seconds of continuous absence, the break counter increases. After 45 minutes of study the recommendation becomes TAKE BREAK. If the person is absent it is PAUSED. If the score is below 60 while studying, it recommends IMPROVE ENV; otherwise KEEP STUDYING.

Efficiency starts from the environment score. After 30 minutes, 10 points are added. If more than three breaks occur, 10 are subtracted. 80+ is EXCELLENT, 65+ GOOD, 50+ AVERAGE, otherwise LOW.

8. LCD Pages

- Page 1: temperature, humidity and light.
- Page 2: distance and person status.
- Page 3: study time.
- Page 4: environment score and light status.
- Page 5: status and recommendation.
- Page 6: efficiency.

9. Example Output

```

Temperature: 29.30 C
Humidity: 66.00 %
Light: 446
Distance: 5.52 cm
Person: YES
Study Time: 0 min 13 sec
Break Time: 0 min 0 sec
Breaks: 0
Environment Score: 83/100
Status: STUDYING
Recommendation: KEEP STUDYING
Efficiency: EXCELLENT

```

10. Complete Final Code

```

#include <Wire.h>
#include <LiquidCrystal_I2C.h>
#include <DHT.h>

#define DHTPIN D4
#define DHTTYPE DHT11
#define TRIG D5
#define ECHO D6
#define LED D7
#define LDR A0

DHT dht(DHTPIN, DHTTYPE);
LiquidCrystal_I2C lcd(0x27, 16, 2);

float temp = 0, hum = 0, dist = -1;
int light = 0, score = 0, breaks = 0;
bool person = false;
unsigned long studyTime=0, breakTime=0, absentTime=0;
unsigned long lastSec=0, lastRead=0, lastLCD=0;
int page=0;

void readSensors(){
    float t=dht.readTemperature(), h=dht.readHumidity();
    if(!isnan(t)) temp=t;
    if(!isnan(h)) hum=h;

    light=analogRead(LDR);

    digitalWrite(TRIG,LOW);
    delayMicroseconds(2);
    digitalWrite(TRIG,HIGH);
    delayMicroseconds(10);
    digitalWrite(TRIG,LOW);

    long duration=pulseIn(ECHO,HIGH,30000);
    if(duration>0) dist=duration*0.0343/2;

```

```

else dist=-1;

person=(dist>=2 && dist<=100);

if(light<150) digitalWrite(LED,HIGH);
else if(light>250) digitalWrite(LED,LOW);

int ts;
if(temp>=20 && temp<=28) ts=25;
else if((temp>=18 && temp<20)|| (temp>28 && temp<=30)) ts=20;
else if((temp>=15 && temp<18)|| (temp>30 && temp<=33)) ts=10;
else ts=5;

int hs;
if(hum>=40 && hum<=60) hs=25;
else if((hum>=30 && hum<40)|| (hum>60 && hum<=70)) hs=20;
else if((hum>=20 && hum<30)|| (hum>70 && hum<=80)) hs=10;
else hs=5;

int ls=(light>=500)?25:((light>=200)?18:8);
score=ts+hs+ls+(person?25:0);
}

void updateTimer(){
  if(millis()-lastSec>=1000){
    lastSec=millis();
    if(person){ studyTime++; absentTime=0; }
    else { breakTime++; absentTime++; }
    if(absentTime==300) breaks++;
  }
}

String status(){
  if(person) return "STUDYING";
  if(absentTime>=300) return "BREAK";
  return "PAUSED";
}

String recommendation(){
  if(!person) return "PAUSED";
  if(studyTime>=2700) return "TAKE BREAK";
  if(score<60) return "IMPROVE ENV";
  return "KEEP STUDYING";
}

String efficiency(){
  int s=score;
  if(studyTime>=1800) s+=10;
  if(breaks>3) s-=10;
  if(s>=80) return "EXCELLENT";
  if(s>=65) return "GOOD";
  if(s>=50) return "AVERAGE";
  return "LOW";
}

void printTime(unsigned long sec){
  lcd.print(sec/60);
  lcd.print(":");
  if(sec%60<10) lcd.print("0");
  lcd.print(sec%60);
}

void showLCD(){
  lcd.clear();

  if(page==0){
    lcd.setCursor(0,0);
    lcd.print("T:"); lcd.print(temp,1); lcd.print("C H:"); lcd.print(hum,0);
    lcd.setCursor(0,1); lcd.print("Light:"); lcd.print(light);
  }
  else if(page==1){
    lcd.setCursor(0,0); lcd.print("Distance:");
    if(dist>=0) lcd.print(dist,0); else lcd.print("---");
    lcd.print("cm");
    lcd.setCursor(0,1); lcd.print("Person:"); lcd.print(person?"YES":"NO");
  }
  else if(page==2){
    lcd.setCursor(0,0); lcd.print("Study Time");
    lcd.setCursor(0,1); printTime(studyTime);
  }
  else if(page==3){
    lcd.setCursor(0,0); lcd.print("Score:"); lcd.print(score); lcd.print("/100");
    lcd.setCursor(0,1); lcd.print(light<200?"LIGHT:LOW":"LIGHT:GOOD");
  }
  else if(page==4){
    lcd.setCursor(0,0); lcd.print(status());
    lcd.setCursor(0,1); lcd.print(recommendation());
  }
  else{
    lcd.setCursor(0,0); lcd.print("Efficiency");
    lcd.setCursor(0,1); lcd.print(efficiency());
  }
}

void report(){
  Serial.println("\n-----");
  Serial.print("Temperature: "); Serial.print(temp); Serial.println(" C");
  Serial.print("Humidity: "); Serial.print(hum); Serial.println(" %");
  Serial.print("Light: "); Serial.println(light);
  Serial.print("Distance: ");
  if(dist>=0){Serial.print(dist);Serial.println(" cm");}
  else Serial.println("NO ECHO");
  Serial.print("Person: "); Serial.println(person?"YES":"NO");
  Serial.print("Study Time: "); Serial.print(studyTime/60);
}

```

```

Serial.print(" min "); Serial.print(studyTime%60); Serial.println(" sec");
Serial.print("Break Time: "); Serial.print(breakTime/60);
Serial.print(" min "); Serial.print(breakTime%60); Serial.println(" sec");
Serial.print("Breaks: "); Serial.println(breaks);
Serial.print("Environment Score: "); Serial.print(score); Serial.println("/100");
Serial.print("Status: "); Serial.println(status());
Serial.print("Recommendation: "); Serial.println(recommendation());
Serial.print("Efficiency: "); Serial.println(efficiency());
Serial.println("-----");
}

void setup(){
  Serial.begin(9600);
  dht.begin();

  pinMode(TRIG,OUTPUT);
  pinMode(ECHO,INPUT);
  pinMode(LED,OUTPUT);
  digitalWrite(LED,LOW);

  Wire.begin(D2,D1);
  lcd.init();
  lcd.backlight();

  lcd.clear();
  lcd.setCursor(0,0); lcd.print("SMART STUDY");
  lcd.setCursor(0,1); lcd.print("SYSTEM READY");
  delay(2000);

  readSensors();
  showLCD();

  lastSec=millis();
  lastRead=millis();
  lastLCD=millis();
}

void loop(){
  unsigned long now=millis();

  if(now-lastRead>=2000){
    lastRead=now;
    readSensors();
    report();
  }

  updateTimer();

  if(now-lastLCD>=3000){
    lastLCD=now;
    page++;
    if(page>5) page=0;
    showLCD();
  }
}

```

11. Important Code Explanation

- #define gives readable names to pins and sensor settings.
- readSensors() reads DHT11, LDR and HC-SR04, controls the LED and calculates the score.
- updateTimer() uses millis() so the system can count seconds without blocking the whole program.
- status(), recommendation() and efficiency() convert raw values into understandable messages.
- showLCD() cycles through six LCD pages.
- report() prints a complete diagnostic report to Serial Monitor.
- setup() initializes hardware and shows the startup message.
- loop() repeatedly performs sensor reading, timing and LCD updates.

12. Vayavya Labs Viva Questions and Answers

What is your project title?

Smart Study Environment Monitoring & Analysis System using ESP8266.

Why did you choose this project?

To solve a practical problem by combining environmental monitoring, presence detection, timing and embedded decision logic.

What is the role of ESP8266?

It is the main controller that reads sensors, processes data and controls outputs.

Why DHT11?

It is a low-cost sensor for temperature and humidity.

Why LDR?

It provides an analog indication of light intensity.

Why HC-SR04?

It measures distance and can be used for nearby-person detection.

Why I2C LCD?

It needs only SDA and SCL, saving GPIO pins.

Why resistor with LED?

To limit current and protect the LED and GPIO.

What is A0?

The analog input used to read the LDR signal.

What does pulseIn() do?

It measures the duration of a digital pulse, used here for the ultrasonic echo.

Why millis()?

It enables elapsed-time measurement without long blocking delays.

Why isnan()?

DHT readings can occasionally be invalid, so invalid values are ignored.

What is the maximum score?

100.

How is person detected?

A measured distance from 2 to 100 cm is treated as presence.

What happens when the person leaves?

Study time stops increasing and absence/break time increases.

When is a break counted?

After 300 seconds of continuous absence.

How can this project be improved?

Add Wi-Fi/cloud logging, mobile dashboard, RTC, data storage, better sensors and personalized analytics.

What was a key debugging method?

Testing the LED separately first proved the D7 circuit before integrating it with the full sensor program.

13. One-Minute Interview Explanation

“My project is a Smart Study Environment Monitoring and Analysis System using ESP8266. It monitors the study environment and provides useful feedback. I used DHT11 for temperature and humidity, an LDR for light intensity, and HC-SR04 for person detection. The ESP8266 processes these values, calculates an environment score out of 100, tracks study and break time, controls an LED for low-light indication, and displays the information on a 16×2 I2C LCD. The project helped me apply Embedded C/C++ concepts such as functions, conditions, GPIO, analog input, sensor interfacing, serial communication and non-blocking timing using millis().”

14. Troubleshooting

- DHT11 READ FAILED → check DATA, VCC, GND and DHT type/pin.
- LCD backlight but no text → check I2C address, SDA/SCL and contrast.
- Distance NO ECHO → check TRIG/ECHO wiring and power.
- LED not glowing → check polarity and resistor; test D7 separately.
- LED rapidly blinking → use the hysteresis thresholds in this final code.
- Unreadable Serial output → set Serial Monitor to 9600 baud.

15. Future Scope

- Wi-Fi/cloud data logging.
- Mobile/web dashboard.
- Daily and weekly study graphs.
- RTC for real date/time.
- More accurate sensors such as DHT22/BME280.
- Automatic lighting/fan control with safe driver circuits.
- Personalized study recommendations using stored data.

16. Final Demonstration Checklist

- Board: NodeMCU 1.0 (ESP-12E).
- Serial Monitor: 9600 baud.
- DHT11 DATA → D4.
- HC-SR04 TRIG → D5, ECHO → D6.
- LED + 1 kΩ resistor → D7.
- LDR → A0.
- LCD SDA → D2, SCL → D1.
- Common GND.
- LCD address 0x27 if that is your working address.
- Be able to explain score, timer, LED and sensor logic.

17. Final Takeaway

For an interview, do not memorize every line. Understand the data flow: sensors → ESP8266 → processing → score/status → LCD/LED/Serial. Be ready to explain why each component was selected, how the code works, what you tested, what problems occurred, and how you solved them.